Exercise: Structured Extraction from Semi-Structured Data with AI and ETL

Objective

Your task is to process a semi-structured dataset (attached JSON file) and extract key information into a structured, canonical format. The final structured data should be exposed as a simple RESTful API.

Challenges

1. **Data Transformation**: Extract and map key information from an unstructured/semi-structured JSON dataset.
2. **ETL Pipeline**: Implement a robust transformation process to ensure data consistency.
3. **Generative AI**: Use LLMs or AI-based methods to assist in extracting eligibility requirements and benefit details.
4. **API Exposure**: Implement a simple RESTful API to expose the structured data.

Dataset

You are provided with a JSON file (`dupixent.json`) that contains details about the **Dupixent MyWay Copay Card**. The document has a mix of structured fields and free-text eligibility details. Your task is to:

- Extract structured information.
- Infer missing details using rule-based transformations or generative AI.
- Standardize the format according to the output example.

Expected Output Structure

Your transformed output should follow this structured format:

```
Unset
{
    "program_name": "Dupixent MyWay Copay Card",
    "coverage_eligibilities": ["Commercially insured"],
    "program_type": "Coupon",
    "requirements": [
        {
            "name": "us_residency",
            "value": "true"
```

```
        },
        {
            "name": "minimum_age",
            "value": "18"
        },
        {
            "name": "insurance_coverage",
            "value": "true"
        },
        {
            "name": "eligibility_length",
            "value": "12m"
        }
    ],
    "benefits": [
        {
            "name": "max_annual_savings",
            "value": "13000.00"
        },
        {
            "name": "min_out_of_pocket",
            "value": "0.00"
        }
    ],
    "forms": [
        {
            "name": "Enrollment Form",
            "link": "https://www.dupixent.com/support-savings/copay-card"
        }
    ],
    "funding": {
        "evergreen": "true",
        "current_funding_level": "Data Not Available"
    },
    "details": [
        {
            "eligibility": "Patient must have commercial insurance and be a
legal resident of the US",
            "program": "Patients may pay as little as $0 for every month of
Dupixent",
            "renewal": "Automatically re-enrolled every January 1st if used
within 18 months",
            "income": "Not required"
        }
```

```
    ]
}
```

Implementation Requirements

Step 1: Extract and Clean Data

- Read the provided JSON file and identify key fields.
- Extract and normalize fields like **Program Name, Coverage Eligibility, Program Type, and Benefits**.
- Ensure currency fields (`AnnualMax`) are properly formatted.

Step 2: Extract Eligibility & Requirements Using AI

- Use generative AI to parse and summarize the **EligibilityDetails** text field into structured requirements.
- Convert **free-text eligibility conditions** into structured JSON key-value pairs.

Step 3: Standardize Data for API Exposure

- Ensure consistent data types (e.g., `true/false`, numbers as strings).
- Implement validation rules for missing or ambiguous data.

Step 4: Expose as a RESTful API

- Create a simple Flask (Python) or Express (TypeScript) API that:
    - Provides an endpoint (`/programs/<program_id>`) returning structured JSON.
    - Supports querying program details dynamically.

Bonus Challenges

1. **AI Extraction Improvement**:

    - Use an **LLM** (GPT, Claude, etc.) to extract more nuanced eligibility criteria.
    - For example, detect **age limits**, **geographic restrictions**, or **insurance conditions** from free-text fields.

2. **Data Enrichment**:

- Implement additional **rules-based logic** for missing fields.
- Example: If **expiration date** is missing, assume **default end-of-year**.

3. **Performance Optimization**:

    - Preprocess and **cache structured data** for API efficiency.
    - Allow filtering results dynamically based on parameters.

Deliverables

1. **Python or TypeScript code** implementing:

    - ETL pipeline for structured extraction.
    - AI-powered eligibility extraction.
    - A minimal API to serve structured data.

2. **README with instructions** on:

    - How to run the ETL pipeline.
    - How to start the API and test it.

Evaluation Criteria

- **Correctness:** Extracted data aligns with the expected schema.
- **AI Integration:** Uses AI effectively for text summarization.
- **Code Readability & Modularity:** Clean, structured, and well-documented.
- **API Design:** RESTful, scalable, and efficient.
- **Performance Considerations:** Optimized extraction and API queries.

Submission Instructions

- Submit your code via **GitHub repo** or **zip file**.
- Include sample API responses.
- (Optional) Include **unit tests** for key components.

Good luck! 🚀